LinkedList — Complete Guide

Enhanced Beginner Edition — original content preserved, with added plain-English explanations, brute-force reasoning, and "why we optimize" insight for every problem.

95. Introduction to LinkedList

Understanding the Problem NEW

A LinkedList is a completely different way of storing a sequence of values compared to an array. Instead of sitting in one contiguous block of memory (where you can jump straight to index 5 with simple arithmetic), each element ('node') is its own separate little box containing a value *and* a pointer/reference to the next box. To get to the 5th element, you must walk from the first box, through the second, third, and fourth, following the 'next' pointers each time.

Brute Force Approach NEW

There's no brute-vs-optimized story for the concept itself — this section is purely about understanding the building block. But it's worth contrasting directly with arrays: an array lets you jump to any index in O(1) (direct memory address calculation), while a linked list can only be walked one step at a time, O(n) to reach an arbitrary position.

Why We Optimize NEW

So why use a linked list at all if array access is faster? Because linked lists make **inserting and deleting** elements (especially at the front, or in the middle once you already have a reference to the right spot) an O(1) operation — you just rewire a couple of pointers, with no need to shift every subsequent element like an array requires. This is the fundamental trade-off to keep in mind for the rest of this chapter: linked lists trade away fast random access in exchange for fast insertion/deletion.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Understanding LinkedList structure and basic operations.

Visual Diagram

```
LinkedList Node Structure:
          ■■■■■■■■■■■■■■■■■■■■
          ■ data ■ next ■      ■
          ■■■■■■■■■■■■■■■■■■■■
          ■   Node Structure ■
```

```
          ■■■■■■■■■■■■■■■■■■

          Example: [1] → [2] → [3] → null

                ↑        ↑       ↑
              Head     Node2   Node3

       Singly Linked List:
       Head → [10] → [20] → [30] → null

       Traversal:
       Start at Head, follow next pointers until null
```

Solution Approach

Concept: Node contains data and reference to next node.

Java Implementation

```
// Node class
       class Node {
           int data;
           Node next;

           Node(int data) {
               this.data = data;
               this.next = null;
           }
       }

       // LinkedList class
       class LinkedList {
           Node head;

           // Initialize empty list
           LinkedList() {
               this.head = null;
           }
       }
```

Time & Space Complexity

Time Complexity: O(1) | Space Complexity: O(1)

96. Inserting a Node in LinkedList

Understanding the Problem NEW

Insert a new node into a linked list — at the beginning, at the end, or at a specific position in the middle.

Brute Force Approach NEW

Without linked lists, a beginner used to arrays might imagine you'd need to 'shift everything over' to make room for a new element — that's exactly the O(n) cost arrays pay for insertion.

Why We Optimize NEW

Because a linked list is just a chain of pointers, inserting anywhere only requires updating **at most two pointers**: the new node's 'next' pointer (pointing it at whatever came after the insertion point), and the previous node's 'next' pointer (pointing it at the new node instead). No shifting of existing elements is needed at all. Inserting at the very front or with a direct reference to the insertion point is O(1); inserting at an arbitrary *position* (by index) still costs O(n) just to walk there first, but the insertion itself, once you've arrived, is instant.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Insert node at beginning, end, or specific position.

Visual Diagram

```
Original: [10] → [20] → [30] → null

       Insert at beginning (5):
       Step 1: Create new node [5]
       Step 2: new.next = head
       Step 3: head = new
       Result: [5] → [10] → [20] → [30] → null

       Insert at end (40):
       Step 1: Create new node [40]
       Step 2: Traverse to end
       Step 3: last.next = new
```

```
Result: [10] → [20] → [30] → [40] → null

Insert at position 2 (value 25):
Step 1: Create new node [25]
Step 2: Traverse to position 1
Step 3: new.next = temp.next
Step 4: temp.next = new
Result: [10] → [25] → [20] → [30] → null
```

Solution Approach

Concept: Handle all insertion cases with proper pointer updates.

Algorithm

```
Algorithm INSERT_AT_BEGINNING(head, data):
           new_node = Node(data)
           new_node.next = head
           Return new_node

       Algorithm INSERT_AT_END(head, data):
           new_node = Node(data)
           If head == null: Return new_node
           current = head
           While current.next != null:
               current = current.next
           current.next = new_node
           Return head
```

Java Implementation

```
class LinkedList {
           Node head;

            // Insert at beginning
            public void insertAtBeginning(int data) {
                Node newNode = new Node(data);
                newNode.next = head;
                head = newNode;
            }

            // Insert at end
            public void insertAtEnd(int data) {

               Node newNode = new Node(data);
               if (head == null) {
                   head = newNode;
                   return;
               }
               Node current = head;
```

```
            while (current.next != null) {
                current = current.next;
            }
            current.next = newNode;
        }

        // Insert at specific position (0-based)
        public void insertAtPosition(int data, int position) {
            Node newNode = new Node(data);
            if (position == 0) {
                newNode.next = head;
                head = newNode;
                return;
            }

            Node current = head;
            for (int i = 0; i < position - 1 && current != null; i++) {
                current = current.next;
            }

            if (current == null) return;
            newNode.next = current.next;
            current.next = newNode;
        }
    }
```

Time & Space Complexity

Time Complexity: O(1) at beginning, O(n) at end | Space Complexity: O(1)

97. Deleting a Node in LinkedList

Understanding the Problem NEW

Delete a node from a linked list — from the beginning, the end, or a specific position — handling the tricky edge cases (deleting the head, deleting the only node, deleting the tail) correctly.

Brute Force Approach NEW

An array-based mental model would suggest 'shift everything after the deleted position back by one' — O(n) work, again unnecessary for a linked list.

Why We Optimize NEW

Deletion in a linked list just means **re-pointing** the previous node's 'next' to skip over the node being removed (so it points directly to what came after it) — the deleted node simply becomes unreachable and gets cleaned up. This is O(1) once you have a reference to the node just before the one you're deleting; the only O(n) cost is walking to find that position in the first place if you're given an index rather than a direct pointer.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Delete node from beginning, end, or specific position.

Visual Diagram

```
Original: [10] → [20] → [30] → [40] → null

       Delete from beginning:
       Step 1: head = head.next
       Result: [20] → [30] → [40] → null

       Delete from end:
       Step 1: Traverse to second last
       Step 2: secondLast.next = null
       Result: [10] → [20] → [30] → null

       Delete at position 2 (delete 30):
       Step 1: Traverse to position 1 (20)
```

```
       Step 2: temp.next = temp.next.next
       Result: [10] → [20] → [40] → null
```

Solution Approach

Concept: Handle edge cases and update pointers.

Algorithm

```
Algorithm DELETE_FROM_BEGINNING(head):
           If head == null: Return null
           Return head.next

       Algorithm DELETE_FROM_END(head):
           If head == null: Return null
           If head.next == null: Return null
           current = head
           While current.next.next != null:
               current = current.next
           current.next = null
           Return head
```

Java Implementation

```
class LinkedList {
           Node head;

           // Delete from beginning
           public void deleteFromBeginning() {
               if (head == null) return;
               head = head.next;
           }

           // Delete from end
           public void deleteFromEnd() {
               if (head == null) return;
               if (head.next == null) {
                   head = null;
                   return;
               }
               Node current = head;
               while (current.next.next != null) {

                   current = current.next;
               }
               current.next = null;
           }

           // Delete at specific position
           public void deleteAtPosition(int position) {
```

```
            if (head == null) return;
            if (position == 0) {
                head = head.next;
                return;
            }

            Node current = head;
            for (int i = 0; i < position - 1 && current.next != null; i++) {
                current = current.next;
            }

            if (current.next == null) return;
            current.next = current.next.next;
        }
    }
```

Time & Space Complexity

Time Complexity: O(1) at beginning, O(n) at end | Space Complexity: O(1)

98. Find the Length of the LinkedList

Understanding the Problem NEW

Count how many nodes are in the linked list.

Brute Force Approach NEW

There isn't a faster alternative here — a linked list has no built-in 'length' field by default (unlike some language's array types), so counting requires visiting every node.

Why We Optimize NEW

Since there's no way to jump ahead or skip nodes, you simply walk from the head to the end, incrementing a counter at each node, until you reach a null 'next' pointer — an unavoidable O(n) traversal. (Some linked list implementations choose to maintain a running length counter that's updated on every insert/delete, turning this into an O(1) lookup at the cost of a little bookkeeping on every mutation — a classic time-vs-maintenance trade-off worth knowing about.)

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count number of nodes in LinkedList.

Visual Diagram

```
List: [10] → [20] → [30] → [40] → null

       Traverse:
       current = head (10)
       count = 1

       current = 20
       count = 2

       current = 30
       count = 3

       current = 40
       count = 4
```

```
current = null
Result: 4

Empty List:
head = null
count = 0
Result: 0
```

Solution Approach

Concept: Traverse and count nodes.

Algorithm

```
Algorithm LENGTH(head):
           count = 0
           current = head
           While current != null:
               count++
               current = current.next
           Return count
```

Java Implementation

```
class LinkedList {
           Node head;

           public int length() {
               int count = 0;
               Node current = head;
               while (current != null) {
                   count++;
                   current = current.next;
               }
               return count;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

99. Search an Element in the LL

Understanding the Problem NEW

Check whether a given value exists anywhere in the linked list.

Brute Force Approach NEW

As with counting, there's no shortcut available purely from the data structure itself — you must inspect nodes one at a time since there's no way to 'jump' to check a specific value without visiting the nodes in between.

Why We Optimize NEW

Walk from the head, comparing each node's value to the target, and stop the moment you find a match (or reach the end without finding one) — O(n) in the worst case, same as Linear Search on an array. The lesson here (echoing problem 15 in the Arrays chapter) is that unsorted, unstructured data — whether stored as an array or a linked list — fundamentally requires an O(n) search unless you build extra structure (like a hashmap) on top of it.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find if element exists in LinkedList.

Visual Diagram

```
List: [10] → [20] → [30] → [40] → null

       Search for 30:
       current = 10 → not match
       current = 20 → not match
       current = 30 → match ✓
       Return true

       Search for 25:
       current = 10 → not match
       current = 20 → not match
       current = 30 → not match
       current = 40 → not match
```

```
        current = null → end
        Return false
```

Solution Approach

Concept: Traverse and compare each node's data.

Algorithm

```
Algorithm SEARCH(head, target):
            current = head
            While current != null:
                If current.data == target:
                    Return true
                current = current.next
            Return false
```

Java Implementation

```
class LinkedList {
            Node head;

             public boolean search(int target) {
                 Node current = head;
                 while (current != null) {
                     if (current.data == target) {
                         return true;
                     }
                     current = current.next;
                 }
                 return false;
             }
        }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

100. Introduction to DLL

Understanding the Problem NEW

A Doubly LinkedList (DLL) is like a singly linked list, but each node also keeps a pointer **back** to the previous node, not just forward to the next one. This lets you traverse in both directions.

Brute Force Approach NEW

Nothing to optimize here conceptually — it's a structural extension. Worth noting the trade-off though: a DLL uses extra memory (one more pointer per node) in exchange for new capabilities.

Why We Optimize NEW

The extra 'prev' pointer unlocks operations that are awkward or impossible efficiently in a singly linked list — like deleting a node when you're only given a reference to that node itself (without needing to first walk from the head to find its predecessor), or traversing backward from the tail. This is why several later problems in this chapter (delete, reverse, remove duplicates) become simpler or faster on a DLL than on a singly linked list.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Understanding Doubly LinkedList structure.

Visual Diagram

```
Doubly Node Structure:
       ■■■■■■■■■■■■■■■■■■■■■■■■■■■
       ■ prev ■ data ■ next ■    ■
       ■■■■■■■■■■■■■■■■■■■■■■■■■■■
       ■    DLL Node Structure ■
       ■■■■■■■■■■■■■■■■■■■■■■■■■■■

       Example: null ← [10] ↔ [20] ↔ [30] → null
                       ↑      ↑       ↑
                     Head   Node2  Node3

       Traversal:
```

```
       Forward: Start at Head, follow next
       Backward: Start at Tail, follow prev
```

Solution Approach

Concept: Node has prev and next pointers.

Java Implementation

```
// Node class for DLL
       class DLLNode {
           int data;
           DLLNode prev;
           DLLNode next;

           DLLNode(int data) {
               this.data = data;
               this.prev = null;
               this.next = null;
           }
       }

       // DoublyLinkedList class
       class DoublyLinkedList {
           DLLNode head;
           DLLNode tail;

           DoublyLinkedList() {
               this.head = null;
               this.tail = null;
           }
       }
```

101. Insert a Node in DLL

Understanding the Problem NEW

Insert a node into a Doubly LinkedList, correctly wiring up both the 'next' pointer of the preceding node *and* the 'prev' pointer of the following node.

Brute Force Approach NEW

A common beginner mistake is to update only the 'next' pointers (as you would in a singly linked list) and forget the 'prev' pointers entirely, silently corrupting the backward-traversal capability of the list.

Why We Optimize NEW

Every insertion in a DLL requires updating **up to four** pointer connections instead of two: the new node's next and prev, plus the neighbours' pointers pointing back at the new node. Getting the order of these pointer updates right (so you don't accidentally lose a reference to a node before you've rewired everything) is the key skill here — still O(1) once you're at the right position, just with twice the bookkeeping of a singly linked list.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Insert node in Doubly LinkedList.

Visual Diagram

```
Original: null ← [10] ↔ [20] ↔ [30] → null

       Insert at beginning (5):
       Step 1: new.next = head
       Step 2: head.prev = new
       Step 3: head = new
       Step 4: head.prev = null
       Result: null ← [5] ↔ [10] ↔ [20] ↔ [30] → null

       Insert at end (40):
       Step 1: new.prev = tail
       Step 2: tail.next = new
       Step 3: tail = new
```

```
       Result: null ← [10] ↔ [20] ↔ [30] ↔ [40] → null

       Insert after node with value 20 (25):
       Step 1: Create new node [25]
       Step 2: new.next = current.next
       Step 3: new.prev = current
       Step 4: current.next.prev = new
       Step 5: current.next = new
       Result: null ← [10] ↔ [20] ↔ [25] ↔ [30] → null
```

Solution Approach

Concept: Handle prev and next pointers carefully.

Algorithm

```
Algorithm INSERT_IN_DLL(head, data, position):
           new_node = Node(data)
           If position == 0:
               new_node.next = head
               If head != null: head.prev = new_node
               Return new_node

           current = head
           For i = 0 to position-1:
               current = current.next
               If current == null: Return head

           new_node.prev = current
           new_node.next = current.next
           If current.next != null: current.next.prev = new_node
           current.next = new_node
           Return head
```

Java Implementation

```
class DoublyLinkedList {
           DLLNode head;
           DLLNode tail;

           // Insert at beginning
           public void insertAtBeginning(int data) {

               DLLNode newNode = new DLLNode(data);
               if (head == null) {
                   head = tail = newNode;
                   return;
               }
               newNode.next = head;
               head.prev = newNode;
```

```
            head = newNode;
        }

        // Insert at end
        public void insertAtEnd(int data) {
            DLLNode newNode = new DLLNode(data);
            if (tail == null) {
                head = tail = newNode;
                return;
            }
            newNode.prev = tail;
            tail.next = newNode;
            tail = newNode;
        }

        // Insert after specific node
        public void insertAfter(DLLNode prevNode, int data) {
            if (prevNode == null) return;
            DLLNode newNode = new DLLNode(data);
            newNode.next = prevNode.next;
            newNode.prev = prevNode;
            if (prevNode.next != null) {
                prevNode.next.prev = newNode;
            } else {
                tail = newNode;
            }
            prevNode.next = newNode;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) for position, O(1) at ends | Space Complexity: O(1)

102. Delete a Node in DLL

Understanding the Problem NEW

Delete a node from a Doubly LinkedList, correctly handling the head, the tail, and middle nodes.

Brute Force Approach NEW

As above, forgetting to update the 'prev' pointer of the node *after* the deleted one (not just the 'next' pointer of the node before it) is the classic mistake.

Why We Optimize NEW

Because you have direct access to both neighbours via the deleted node's own prev/next pointers, you can splice it out by connecting those two neighbours directly to each other (in both directions) — O(1), and notably, unlike a singly linked list, you don't need a separate reference to the 'previous' node walked in from the head, since the node being deleted already carries that reference itself. This is the concrete payoff of the extra memory a DLL uses.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Delete node from Doubly LinkedList.

Visual Diagram

```
Original: null ← [10] ↔ [20] ↔ [30] ↔ [40] → null

       Delete first node (10):
       Step 1: head = head.next
       Step 2: head.prev = null
       Result: null ← [20] ↔ [30] ↔ [40] → null

       Delete last node (40):
       Step 1: tail = tail.prev
       Step 2: tail.next = null
       Result: null ← [10] ↔ [20] ↔ [30] → null

       Delete node with value 20:
       Step 1: current.next.prev = current.prev
```

```
       Step 2: current.prev.next = current.next
       Result: null ← [10] ↔ [30] ↔ [40] → null
```

Solution Approach

Concept: Handle all cases including head and tail.

Algorithm

```
Algorithm DELETE_IN_DLL(head, node):
           If head == null or node == null: Return head
           If node == head: head = head.next
           If node.next != null: node.next.prev = node.prev
           If node.prev != null: node.prev.next = node.next
           Return head
```

Java Implementation

```
class DoublyLinkedList {
           DLLNode head;
           DLLNode tail;

           // Delete first node
           public void deleteFirst() {
               if (head == null) return;
               if (head == tail) {
                   head = tail = null;
                   return;
               }
               head = head.next;
               head.prev = null;
           }

           // Delete last node
           public void deleteLast() {
               if (tail == null) return;
               if (head == tail) {
                   head = tail = null;
                   return;
               }
               tail = tail.prev;

               tail.next = null;
           }

           // Delete specific node
           public void deleteNode(DLLNode node) {
               if (node == null) return;
               if (node == head) {
                   deleteFirst();
```

```
                return;
            }
            if (node == tail) {
                deleteLast();
                return;
            }
            node.prev.next = node.next;
            node.next.prev = node.prev;
        }
    }
```

Time & Space Complexity

Time Complexity: O(1) if node known, O(n) if search needed | Space Complexity:

103. Reverse a DLL

Understanding the Problem NEW

Reverse a Doubly LinkedList so that traversing it forward now visits nodes in the opposite order.

Brute Force Approach NEW

A beginner might try to build an entirely new reversed list by reading the original front-to-back and inserting new nodes at the front of a fresh list — this works but needlessly allocates a whole new set of nodes when the existing ones can simply be rewired.

Why We Optimize NEW

Since every node already has both 'next' and 'prev' pointers, reversing the whole list is just a matter of **swapping next and prev on every single node** — what used to be 'forward' becomes 'backward' and vice versa — then updating the head/tail references. This is done in-place in a single O(n) pass with no new memory allocation, exploiting the fact a DLL already has all the pointer information needed in both directions.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Reverse the Doubly LinkedList.

Visual Diagram

```
Original: null ← [10] ↔ [20] ↔ [30] → null
       Reverse: null ← [30] ↔ [20] ↔ [10] → null

       Step-by-step:
       Step 1: current = head (10)
       Step 2: Swap prev and next of each node
               [10] prev=null, next=null → prev=20, next=null
       Step 3: Move to next node
               [20] prev=10, next=30 → prev=30, next=10
       Step 4: Continue...
       Result: null ← [30] ↔ [20] ↔ [10] → null
```

Solution Approach

Concept: Swap prev and next for each node.

Algorithm

```
Algorithm REVERSE_DLL(head):
           current = head
           temp = null

           While current != null:
               temp = current.prev
               current.prev = current.next
               current.next = temp
               current = current.prev

           If temp != null:
               head = temp.prev

           Return head
```

Java Implementation

```
class DoublyLinkedList {
           DLLNode head;

           public void reverse() {
               DLLNode current = head;
               DLLNode temp = null;

                 while (current != null) {
                     temp = current.prev;
                     current.prev = current.next;
                     current.next = temp;
                     current = current.prev;
                 }

                 if (temp != null) {
                     head = temp.prev;
                 }
           }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

104. Middle of a LinkedList (Tortoise & Hare)

Understanding the Problem NEW

Find the middle node of a singly linked list in a single pass, without first counting the total length.

Brute Force Approach NEW

The obvious two-pass approach: first traverse the whole list once just to count its length, then traverse again to the length/2 position — correct, but two full passes when one might suffice.

Why We Optimize NEW

The classic **Tortoise and Hare** trick: use two pointers starting at the head, one ('slow') moving one step at a time, the other ('fast') moving two steps at a time. By the time 'fast' reaches the end, 'slow' will be exactly at the middle — because fast covers ground twice as quickly. This finds the middle in a single O(n) pass instead of two, and this same two-speed-pointer idea reappears repeatedly later in this chapter (cycle detection, palindrome checking).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the middle node of a LinkedList.

Visual Diagram

```
List: [1] → [2] → [3] → [4] → [5] → null

       Tortoise (slow) moves 1 step, Hare (fast) moves 2 steps:
       Step 1: slow=1, fast=1
       Step 2: slow=2, fast=3
       Step 3: slow=3, fast=5
       Step 4: slow=4, fast=null (stop)
       Result: slow=4 (middle)

       List: [1] → [2] → [3] → [4] → [5] → [6] → null
       Step 1: slow=1, fast=1
       Step 2: slow=2, fast=3
       Step 3: slow=3, fast=5
```

```
       Step 4: slow=4, fast=null (stop)
       Result: slow=4 (second middle for even length)
```

Solution Approach

Concept: Two pointers moving at different speeds.

Algorithm

```
Algorithm MIDDLE(head):
           slow = head
           fast = head

           While fast != null and fast.next != null:
               slow = slow.next
               fast = fast.next.next

           Return slow
```

Java Implementation

```
class Solution {
           public Node middleNode(Node head) {
               Node slow = head;
               Node fast = head;

                 while (fast != null && fast.next != null) {
                     slow = slow.next;
                     fast = fast.next.next;
                 }
                 return slow;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

105. Reverse a LinkedList [Iterative]

Understanding the Problem NEW

Reverse a singly linked list in-place, so the last node becomes the new head and every 'next' pointer points backward relative to the original order.

Brute Force Approach NEW

As with the DLL version, a beginner might build a brand-new list by reading the original and prepending nodes to a fresh list — correct, but allocates n new nodes when the existing ones can just be rewired.

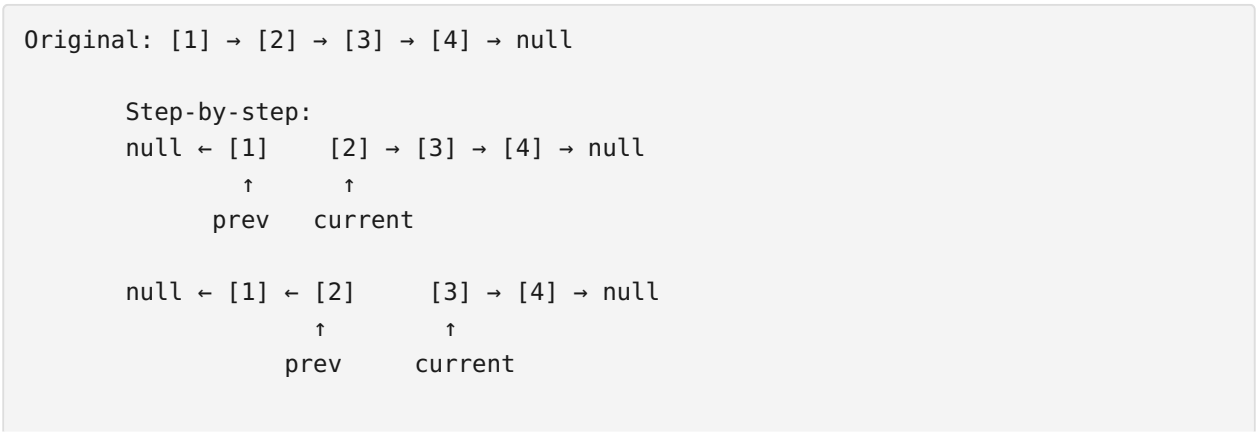
Why We Optimize NEW

Since a singly linked list only has 'next' pointers (no 'prev'), you can't just swap two fields like in the DLL case — instead, you walk through the list with **three pointers**: `prev` (initially null), `current` (starting at head), and a temporary `next` to remember where to go before you overwrite `current`'s pointer. At each step, you redirect `current.next` to point backward at `prev`, then advance all three pointers forward. This reverses the entire list in-place in a single O(n) pass with O(1) extra space — the 'three pointers' pattern here is foundational and reused in problem 122 (reverse in groups).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Reverse LinkedList using iterative approach.

Visual Diagram



```
       null ← [1] ← [2] ← [3]       [4] → null
                          ↑          ↑
                        prev       current

       null ← [1] ← [2] ← [3] ← [4]
                                 ↑
                               head

       Result: [4] → [3] → [2] → [1] → null
```

Solution Approach

Concept: Use three pointers: prev, current, next.

Algorithm

```
Algorithm REVERSE_ITERATIVE(head):
           prev = null
           current = head

           While current != null:
               next = current.next
               current.next = prev
               prev = current
               current = next

           Return prev
```

Java Implementation

```
class Solution {
           public Node reverseList(Node head) {
               Node prev = null;
               Node current = head;

                 while (current != null) {
                     Node next = current.next;
                     current.next = prev;
                     prev = current;
                     current = next;
                 }
                 return prev;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

106. Reverse a LL [Recursive]

Understanding the Problem NEW

Same reversal as above, but implemented recursively instead of with an explicit loop and three pointers.

Brute Force Approach NEW

Nothing 'brute' to contrast against here — it's the same problem as 105, just a different implementation style, useful for practicing recursive thinking on linked structures.

Why We Optimize NEW

The recursive version reverses everything *after* the current node first (via the recursive call), and only then fixes up the current node's own pointer, once the rest of the list is already reversed and its new tail is known. This mirrors the 'recurse first, fix up after' pattern seen in Recursive Insertion Sort back in Chapter 1 — same underlying idea, applied to pointers instead of array values. It still achieves O(n) time, but costs O(n) extra space for the recursion call stack, unlike the iterative version's O(1) space.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Reverse LinkedList using recursion.

Visual Diagram

```
Original: [1] → [2] → [3] → [4] → null

       Recursive calls:
       reverse(1):
         reverse(2):
           reverse(3):
             reverse(4):
               return 4 (base case)

                 head = 4
                 3.next.next = 3 (4.next = 3)
                 3.next = null
```

```
                return 4

            2.next.next = 2 (3.next = 2)
            2.next = null
            return 4

          1.next.next = 1 (2.next = 1)
          1.next = null
          return 4

      Result: [4] → [3] → [2] → [1] → null
```

Solution Approach

Concept: Recursively reverse smaller list.

Algorithm

```
Algorithm REVERSE_RECURSIVE(head):
          If head == null or head.next == null:
              Return head

            newHead = REVERSE_RECURSIVE(head.next)
            head.next.next = head
            head.next = null
            Return newHead
```

Java Implementation

```
class Solution {
          public Node reverseList(Node head) {
              if (head == null || head.next == null) {
                  return head;
              }

                  Node newHead = reverseList(head.next);
                  head.next.next = head;
                  head.next = null;

                  return newHead;
            }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n) recursion stack

107. Detect a Loop in LL

Understanding the Problem NEW

Determine whether a linked list has a cycle (some node's 'next' pointer loops back to an earlier node instead of eventually reaching null).

Brute Force Approach NEW

A beginner might try storing every visited node's reference in a hashset, and check before visiting each node whether it's already been seen — correct, O(n) time, but O(n) extra space to remember every node visited.

Why We Optimize NEW

Floyd's Cycle Detection (the Tortoise and Hare again) solves this with O(1) space: use two pointers moving at different speeds (one step vs two steps) through the list. If there's no cycle, the fast pointer will simply reach the end (null) first. But if there *is* a cycle, the fast pointer will eventually 'lap' the slow pointer and they'll land on the exact same node at the same time — a mathematical guarantee for any cycle length. This gets you the same O(n) time as the hashset approach, but without needing any extra memory.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Detect if LinkedList has a cycle.

Visual Diagram

```
List with cycle: [1] → [2] → [3] → [4] → [5]
                               ↑                ↓
                               [8] ← [7] ← [6]

       Floyd's Cycle Detection:
       Step 1: slow=1, fast=1
       Step 2: slow=2, fast=3
       Step 3: slow=3, fast=5
       Step 4: slow=4, fast=7
       Step 5: slow=5, fast=4 (met at 4)
       → Cycle detected!
```

```
No cycle: [1] → [2] → [3] → [4] → null
slow=1, fast=1
slow=2, fast=3
slow=3, fast=5
fast reaches null → no cycle
```

Solution Approach

Concept: Floyd's cycle detection algorithm.

Algorithm

```
Algorithm HAS_CYCLE(head):
           slow = head
           fast = head

            While fast != null and fast.next != null:
                slow = slow.next
                fast = fast.next.next
                If slow == fast:
                    Return true

            Return false
```

Java Implementation

```
class Solution {
           public boolean hasCycle(Node head) {
               Node slow = head;
               Node fast = head;

                 while (fast != null && fast.next != null) {
                     slow = slow.next;
                     fast = fast.next.next;

                     if (slow == fast) {
                         return true;
                     }
                 }
                 return false;
             }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

108. Find the Starting Point of Loop in LL

Understanding the Problem NEW

Given that a linked list has a cycle (from problem 107), find the exact node where the cycle actually begins — not just detect that one exists.

Brute Force Approach NEW

Once you know a cycle exists, tracking every visited node in a hashset and reporting the first one you see twice would find the start — correct, O(n) time, O(n) space.

Why We Optimize NEW

After the slow/fast pointers meet (as in Floyd's algorithm), there's an elegant mathematical fact: if you now move the **slow pointer back to the head** and advance both the slow and fast pointers *one step at a time* from here, they are guaranteed to meet exactly at the start of the cycle. This relies on the specific distance relationships built up during the initial meeting, and again avoids any extra hashset — still O(n) time, O(1) space, extending Floyd's algorithm rather than falling back to extra memory.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the node where cycle begins.

Visual Diagram

```
List: [1] → [2] → [3] → [4] → [5] → [6]
                            ↑               ↓
                            [9] ← [8] ← [7]

       Step 1: Find meeting point (Floyd's)
       slow=1, fast=1
       slow=2, fast=3
       slow=3, fast=5
       slow=4, fast=7
       slow=5, fast=4
       Meeting point = 4
```

```
       Step 2: Find cycle start
       Initialize slow = head (1)
       Move slow and fast 1 step each
       slow=1, fast=4
       slow=2, fast=5
       slow=3, fast=6
       slow=4, fast=7
       Now slow=fast=4 → Cycle starts at 4!

       Result: Cycle starts at node with value 4
```

Solution Approach

Concept: After meeting, move slow to head, both step one by one.

Algorithm

```
Algorithm CYCLE_START(head):
           If no cycle: Return null

           slow = head, fast = head
           While fast != null and fast.next != null:
               slow = slow.next
               fast = fast.next.next
               If slow == fast:
                   slow = head
                   While slow != fast:
                       slow = slow.next
                       fast = fast.next
                   Return slow
           Return null
```

Java Implementation

```
class Solution {
           public Node detectCycle(Node head) {
               Node slow = head;
               Node fast = head;

                 // Find meeting point
                 while (fast != null && fast.next != null) {
                     slow = slow.next;
                     fast = fast.next.next;

                  if (slow == fast) {
                      // Find cycle start
                      slow = head;
                      while (slow != fast) {
                          slow = slow.next;
                          fast = fast.next;
```

```
                    }
                    return slow;
                }
            }
            return null;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

109. Length of Loop in LL

Understanding the Problem NEW

Given a cyclic linked list, find how many nodes are in the cycle itself (not the whole list, just the looping part).

Brute Force Approach NEW

Store visited nodes with a step counter in a hashset; once you revisit a node, the difference between the current step count and that node's stored step count is the cycle length — O(n) time, O(n) space.

Why We Optimize NEW

Once Floyd's algorithm finds any meeting point inside the cycle (from problem 107), you can compute the cycle's length for free without extra memory: just keep one pointer fixed at the meeting point, advance a second pointer one step at a time, and count steps until it comes back around to the same node. This reuses the meeting point you've already found, giving O(cycle length) additional work and O(1) space, instead of tracking every node visited.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the length of the cycle in LinkedList.

Visual Diagram

```
List: [1] → [2] → [3] → [4] → [5] → [6]
                            ↑               ↓
                            [9] ← [8] ← [7]

       Step 1: Find meeting point (Floyd's)
       Meeting point = 4

       Step 2: Count length
       temp = meeting.next (5)
       count = 1
       temp=5 → count=1
       temp=6 → count=2
       temp=7 → count=3
```

```
temp=8 → count=4
temp=9 → count=5
temp=4 (meeting) → stop
Loop length = 5

Result: Length = 5
```

Solution Approach

Concept: Find meeting point, then count nodes until back.

Algorithm

```
Algorithm LOOP_LENGTH(head):
           If no cycle: Return 0
           slow = head, fast = head

            While fast != null and fast.next != null:
                slow = slow.next
                fast = fast.next.next
                If slow == fast:
                     count = 1
                     temp = slow.next
                     While temp != slow:
                         count++
                         temp = temp.next
                     Return count
            Return 0
```

Java Implementation

```
class Solution {
           public int loopLength(Node head) {
               Node slow = head;
               Node fast = head;

                 while (fast != null && fast.next != null) {
                     slow = slow.next;
                     fast = fast.next.next;

                     if (slow == fast) {
                         int count = 1;

                      Node temp = slow.next;
                      while (temp != slow) {
                          count++;
                          temp = temp.next;
                      }
                      return count;
                  }
```

```
            }
            return 0;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

110. Check if LL is Palindrome or Not

Understanding the Problem NEW

Check whether a linked list reads the same forwards and backwards (like an array palindrome check), but linked lists can't be indexed backward directly the way arrays can.

Brute Force Approach NEW

A beginner might copy all the values into an array first, then check the array for palindrome-ness using two pointers from both ends — correct, O(n) time, but O(n) extra space for the copy.

Why We Optimize NEW

You can do this with O(1) extra space by combining two tricks you've already learned in this chapter: find the middle of the list (problem 104's Tortoise and Hare), **reverse the second half in-place** (problem 105's technique), and then walk the first half and the reversed second half together, comparing values node by node. If you want to restore the list afterward, you can reverse the second half back once the comparison is done. This avoids the O(n) array copy entirely, reusing two techniques already built earlier in the chapter.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if LinkedList is palindrome.

Visual Diagram

```
List: [1] → [2] → [3] → [2] → [1] → null
       Palindrome ✓

       Step 1: Find middle (3)
       Step 2: Reverse second half [2] → [1] → null becomes [1] → [2] → null
       Step 3: Compare first and second half
       [1] = [1] ✓
       [2] = [2] ✓
       Result: true

       List: [1] → [2] → [3] → [4] → null
```

```
Not palindrome ✗

Step 1: Find middle (3)
Step 2: Reverse second half [4] → null becomes [4] → null
Step 3: Compare
[1] ≠ [4] ✗
Result: false
```

Solution Approach

Concept: Find middle, reverse second half, compare.

Algorithm

```
Algorithm IS_PALINDROME(head):
           If head == null: Return true

           // Find middle
           slow = head, fast = head
           While fast.next != null and fast.next.next != null:
               slow = slow.next
               fast = fast.next.next

           // Reverse second half
           second = reverseList(slow.next)
           slow.next = null

           // Compare
           first = head
           while second != null:
               If first.data != second.data: Return false
               first = first.next
               second = second.next

           Return true
```

Java Implementation

```
class Solution {
           public boolean isPalindrome(Node head) {
               if (head == null) return true;

                 // Find middle
                 Node slow = head;

              Node fast = head;
              while (fast.next != null && fast.next.next != null) {
                  slow = slow.next;
                  fast = fast.next.next;
              }
```

```
            // Reverse second half
            Node second = reverseList(slow.next);
            slow.next = null;

            // Compare
            Node first = head;
            Node temp = second;
            while (temp != null) {
                if (first.data != temp.data) return false;
                first = first.next;
                temp = temp.next;
            }

            return true;
        }

        private Node reverseList(Node head) {
            Node prev = null;
            Node current = head;
            while (current != null) {
                Node next = current.next;
                current.next = prev;
                prev = current;
                current = next;
            }
            return prev;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

111. Segregate Odd and Even Nodes in LL

Understanding the Problem NEW

Rearrange a linked list so all nodes originally at odd positions (1st, 3rd, 5th...) come first, followed by all nodes originally at even positions (2nd, 4th, 6th...), preserving their relative order within each group.

Brute Force Approach NEW

A beginner might copy all node values into two separate arrays (odd-position values and even-position values), then rebuild the linked list from those arrays — correct, O(n) time, but O(n) extra space, and it discards the existing nodes unnecessarily.

Why We Optimize NEW

Since you already have direct pointer access to every node, you can do this **in-place** by maintaining two running pointers — one tracking the current tail of the 'odd chain,' one tracking the current tail of the 'even chain' — and re-wiring 'next' pointers as you walk through the original list once, finally connecting the end of the odd chain to the head of the even chain. This achieves O(n) time with O(1) extra space, by rewiring existing nodes instead of copying values into new structures.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Segregate odd-indexed and even-indexed nodes.

Visual Diagram

```
Original: [1] → [2] → [3] → [4] → [5] → null

       Odd positions: 1, 3, 5
       Even positions: 2, 4

       Result: [1] → [3] → [5] → [2] → [4] → null

       Step-by-step:
       Step 1: odd = head (1), even = head.next (2)
       Step 2: oddHead = 1, evenHead = 2
```

```
Step 3: Link odd nodes
1.next = 3
3.next = 5
5.next = null
Step 4: Link even nodes
2.next = 4
4.next = null
Step 5: Combine
5.next = 2
Result: 1 → 3 → 5 → 2 → 4
```

Solution Approach

Concept: Separate odd and even position nodes.

Algorithm

```
Algorithm SEGREGATE_ODD_EVEN(head):
           If head == null: Return null

           odd = head
           even = head.next
           evenHead = even

           While even != null and even.next != null:
               odd.next = even.next
               odd = odd.next
               even.next = odd.next
               even = even.next

           odd.next = evenHead
           Return head
```

Java Implementation

```
class Solution {
           public Node oddEvenList(Node head) {
               if (head == null) return null;

                 Node odd = head;
                 Node even = head.next;
                 Node evenHead = even;

                 while (even != null && even.next != null) {
                     odd.next = even.next;

                  odd = odd.next;
                  even.next = odd.next;
                  even = even.next;
              }
```

```
            odd.next = evenHead;
            return head;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

112. Remove Nth Node from the Back

Understanding the Problem NEW

Remove the N-th node counting from the *end* of the list (not the beginning), ideally in a single pass.

Brute Force Approach NEW

The natural two-pass approach: first traverse the whole list to find its total length L, then traverse again to the (L-N)-th node from the start to perform the deletion — correct, but two full passes.

Why We Optimize NEW

Use two pointers with a fixed **gap of N** between them: first advance one pointer N steps ahead on its own, then move both pointers forward together, one step at a time. When the front pointer reaches the end, the back pointer will be sitting exactly at the node just before the one to delete. This achieves the deletion in a single O(n) pass instead of two — the fixed gap does the work that counting the total length would otherwise require.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Remove the Nth node from the end of LinkedList.

Visual Diagram

```
List: [1] → [2] → [3] → [4] → [5] → null, N=2
       Remove: 2nd from end = 4

       Step 1: fast = head
       Move fast N steps: fast = 3

       Step 2: slow = head
       Move both until fast reaches end
       slow=1, fast=3
       slow=2, fast=4
       slow=3, fast=5
       slow=4, fast=null
```

```
Step 3: slow is at node before target
Remove target: slow.next = slow.next.next
Result: [1] → [2] → [3] → [5] → null

N=1 (remove last):
fast = 1 → move to 5
slow = 1 → move to 4
Remove: 4.next = null
Result: [1] → [2] → [3] → [4] → null
```

Solution Approach

Concept: Two pointers with N gap.

Algorithm

```
Algorithm REMOVE_NTH_FROM_END(head, n):
           dummy = Node(0)
           dummy.next = head
           slow = dummy
           fast = dummy

           For i = 1 to n+1:
               fast = fast.next

           While fast != null:
               slow = slow.next
               fast = fast.next

           slow.next = slow.next.next
           Return dummy.next
```

Java Implementation

```
class Solution {
           public Node removeNthFromEnd(Node head, int n) {
               Node dummy = new Node(0);
               dummy.next = head;
               Node slow = dummy;
               Node fast = dummy;

                 // Move fast n+1 steps ahead

              for (int i = 0; i <= n; i++) {
                  fast = fast.next;
              }

              // Move both until fast reaches end
              while (fast != null) {
                  slow = slow.next;
```

```
                fast = fast.next;
            }

            // Remove nth node
            slow.next = slow.next.next;
            return dummy.next;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

113. Delete the Middle Node of LL

Understanding the Problem NEW

Delete the middle node of a linked list directly, without first computing the list's length.

Brute Force Approach NEW

Just like the earlier length-then-traverse approach, you could count nodes first (one pass), then traverse again to the middle position for deletion (a second pass).

Why We Optimize NEW

Reuse the Tortoise and Hare technique from problem 104 (with a small pointer-start adjustment) to locate the middle in a single pass, while also keeping a reference to the node just *before* the middle so you can immediately unlink it the moment you arrive — combining 'find the middle' and 'delete a node' into one O(n) traversal instead of two separate ones.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Delete the middle node of LinkedList.

Visual Diagram

```
List: [1] → [2] → [3] → [4] → [5] → null
       Middle: 3

       Step 1: Find middle
       slow=1, fast=1
       slow=2, fast=3
       slow=3, fast=5
       slow=4, fast=null
       Middle = 3

       Step 2: Delete middle
       Result: [1] → [2] → [4] → [5] → null

       List: [1] → [2] → [3] → [4] → [5] → [6] → null
```

```
Middle: 3 (for even length, first middle)

Step 1: Find middle
slow=1, fast=1
slow=2, fast=3
slow=3, fast=5
slow=4, fast=null
Middle = 3

Result: [1] → [2] → [4] → [5] → [6] → null
```

Solution Approach

Concept: Use two pointers to find middle.

Algorithm

```
Algorithm DELETE_MIDDLE(head):
           If head == null or head.next == null:
               Return null

           slow = head
           fast = head
           prev = null

           While fast != null and fast.next != null:
               prev = slow
               slow = slow.next
               fast = fast.next.next

           prev.next = slow.next
           Return head
```

Java Implementation

```
class Solution {
           public Node deleteMiddle(Node head) {
               if (head == null || head.next == null) {
                   return null;
               }
               Node slow = head;

              Node fast = head;
              Node prev = null;

              while (fast != null && fast.next != null) {
                  prev = slow;
                  slow = slow.next;
                  fast = fast.next.next;
              }
```

```
            prev.next = slow.next;
            return head;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

114. Sort LinkedList

Understanding the Problem NEW

Sort a linked list into ascending order — but unlike an array, you can't randomly access elements by index, which changes which sorting algorithms are practical here.

Brute Force Approach NEW

You could copy all values into an array, sort the array with any standard algorithm (like Chapter 1's Quick Sort), and rebuild the linked list from the sorted array — correct, O(n log n) time, but O(n) extra space for the copy, and it throws away the existing nodes.

Why We Optimize NEW

Merge Sort turns out to be an exceptionally good fit for linked lists specifically, because merging two sorted lists doesn't require random access — you just walk both lists with pointers and relink nodes as you go, and finding the 'middle' to split the list uses the same Tortoise and Hare trick from problem 104. This lets you sort the linked list in-place (rewiring existing nodes) in O(n log n) time with only O(log n) extra space for the recursion, instead of O(n) space for an array copy — Quick Sort, by contrast, doesn't translate nearly as well to linked lists because its partitioning step benefits from random access that a linked list doesn't offer.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Sort a LinkedList using merge sort.

Visual Diagram

```
List: [4] → [2] → [1] → [3] → null

       Step 1: Find middle → [2]
       Step 2: Split into left and right
       Left: [4] → [2] → null
       Right: [1] → [3] → null
```

```
Step 3: Sort each half
Left: [2] → [4] → null
Right: [1] → [3] → null

Step 4: Merge sorted halves
Compare 2 and 1 → 1
Compare 2 and 3 → 2
Compare 4 and 3 → 3
Append 4
Result: [1] → [2] → [3] → [4] → null
```

Solution Approach

Concept: Merge sort on LinkedList.

Algorithm

```
Algorithm SORT_LL(head):
           If head == null or head.next == null:
               Return head

           // Find middle
           slow = head, fast = head
           While fast.next != null and fast.next.next != null:
               slow = slow.next
               fast = fast.next.next

           // Split
           middle = slow.next
           slow.next = null

           // Sort halves
           left = SORT_LL(head)
           right = SORT_LL(middle)

           // Merge
           Return MERGE(left, right)
```

Java Implementation

```
class Solution {
           public Node sortList(Node head) {
               if (head == null || head.next == null) {
                   return head;
               }

                 // Find middle
                 Node slow = head;

               Node fast = head;
```

```
            while (fast.next != null && fast.next.next != null) {
                slow = slow.next;
                fast = fast.next.next;
            }

            // Split
            Node middle = slow.next;
            slow.next = null;

            // Sort halves
            Node left = sortList(head);
            Node right = sortList(middle);

            // Merge
            return merge(left, right);
        }

        private Node merge(Node l1, Node l2) {
            Node dummy = new Node(0);
            Node current = dummy;

            while (l1 != null && l2 != null) {
                if (l1.data < l2.data) {
                    current.next = l1;
                    l1 = l1.next;
                } else {
                    current.next = l2;
                    l2 = l2.next;
                }
                current = current.next;
            }

            current.next = (l1 != null) ? l1 : l2;
            return dummy.next;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log n) | Space Complexity: O(log n)

115. Sort a LL of 0's, 1's and 2's

Understanding the Problem NEW

Sort a linked list that only contains the values 0, 1, and 2 — the linked-list version of the Dutch National Flag problem from the Arrays chapter (problem 23).

Brute Force Approach NEW

You could count the 0s, 1s, and 2s in one pass, then overwrite each node's *value* in a second pass based on the counts — this works (O(n) time) but relies on being able to freely overwrite values, which isn't always the intent when the goal is to rearrange nodes.

Why We Optimize NEW

A cleaner approach true to the spirit of linked lists: create three separate dummy chains (for 0s, 1s, and 2s), walk through the original list once, and append each node to the appropriate chain by rewiring pointers (no value copying needed) — then splice the three chains together in order at the end. This is a direct pointer-based translation of the Dutch National Flag idea, achieving O(n) time and O(1) extra space (beyond the three temporary head pointers) by moving existing nodes around instead of overwriting their contents.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Sort LinkedList containing 0, 1, 2.

Visual Diagram

```
List: [2] → [1] → [0] → [2] → [1] → [0] → null
       Result: [0] → [0] → [1] → [1] → [2] → [2] → null

       Method 1: Count frequencies
       Count 0s: 2, 1s: 2, 2s: 2
       Build list: 0,0,1,1,2,2

       Method 2: Three pointers (same as Dutch Flag)
       zeroHead = null, oneHead = null, twoHead = null
```

```
       Traverse and add to respective lists
       Then combine: zero → one → two
```

Solution Approach

Concept: Use three separate lists for 0, 1, 2.

Algorithm

```
Algorithm SORT_012(head):
           zero = new Node(0), one = new Node(0), two = new Node(0)
           zeroHead = zero, oneHead = one, twoHead = two

            current = head
            While current != null:
                If current.data == 0:
                    zero.next = current
                    zero = zero.next
                Else If current.data == 1:
                    one.next = current
                    one = one.next
                Else:
                    two.next = current
                    two = two.next
                current = current.next

            zero.next = oneHead.next ? oneHead.next : twoHead.next
            one.next = twoHead.next
            two.next = null

            Return zeroHead.next
```

Java Implementation

```
class Solution {
           public Node sort012(Node head) {
               if (head == null) return null;

                 Node zero = new Node(0);
                 Node one = new Node(0);
                 Node two = new Node(0);
                 Node zeroHead = zero, oneHead = one, twoHead = two;

                 Node current = head;
                 while (current != null) {
                     if (current.data == 0) {

                      zero.next = current;
                      zero = zero.next;
                  } else if (current.data == 1) {
```

```
                    one.next = current;
                    one = one.next;
                } else {
                    two.next = current;
                    two = two.next;
                }
                current = current.next;
            }

            zero.next = (oneHead.next != null) ? oneHead.next : twoHead.next;
            one.next = twoHead.next;
            two.next = null;

            return zeroHead.next;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

116. Find the Intersection Point of Y LinkedList

Understanding the Problem NEW

Two singly linked lists eventually merge into the same nodes, forming a Y shape. Find the exact node where they intersect (not just a node with the same *value*, but literally the same node in memory).

Brute Force Approach NEW

A beginner might compare every node in list A against every node in list B, checking for a matching reference — O(n×m) time, or use a hashset of all of list A's node references and check list B's nodes against it — O(n+m) time but O(n) extra space.

Why We Optimize NEW

Because both lists share the exact same tail once they merge, their lengths only differ in the 'unshared' portion before the merge point. If you first find each list's length, then advance the pointer of the *longer* list by the length difference so both pointers now have the same number of remaining nodes to the end, walking both remaining pointers forward together one step at a time will land them on the intersection node simultaneously. This achieves O(n+m) time with O(1) extra space, avoiding the hashset entirely by aligning the two lists' remaining lengths first.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find where two LinkedLists intersect.

Visual Diagram

```
List A: [1] → [2] → [3]
                            ↓
                            [7] → [8] → [9] → null
                            ↑
       List B: [4] → [5] → [6]

       Intersection node: 7
```

```
Method 1: HashSet
Add all nodes of list A to set
Traverse list B and check if node in set

Method 2: Two pointers
Step 1: Find length difference = 3
Step 2: Move longer list (A) by 3
Step 3: Move both until they meet
Result: Intersection at 7
```

Solution Approach

Concept: Align lengths and traverse together.

Algorithm

```
Algorithm INTERSECTION(headA, headB):
           lenA = length(headA), lenB = length(headB)
           diff = abs(lenA - lenB)

           If lenA > lenB:
               Move headA by diff
           Else:
               Move headB by diff

           While headA != null and headB != null:
               If headA == headB:
                   Return headA
               headA = headA.next
               headB = headB.next

           Return null
```

Java Implementation

```
class Solution {
           public Node getIntersectionNode(Node headA, Node headB) {
               int lenA = getLength(headA);
               int lenB = getLength(headB);

                 while (lenA > lenB) {
                     headA = headA.next;
                     lenA--;
                 }
                 while (lenB > lenA) {
                     headB = headB.next;
                     lenB--;

             }
```

```
            while (headA != null && headB != null) {
                if (headA == headB) {
                    return headA;
                }
                headA = headA.next;
                headB = headB.next;
            }
            return null;
        }

        private int getLength(Node head) {
            int len = 0;
            while (head != null) {
                len++;
                head = head.next;
            }
            return len;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n+m) | Space Complexity: O(1)

117. Add 1 to a Number Represented by LL

Understanding the Problem NEW

A number is represented digit-by-digit as a linked list (e.g. `1 -> 2 -> 3` represents 123). Add 1 to this number and return the updated linked list, correctly handling carries (e.g. `1->9->9` becomes `2->0->0`).

Brute Force Approach NEW

Convert the linked list into an actual integer (or string), add 1 using normal arithmetic, then rebuild a new linked list from the result — works for small numbers, but breaks down for very large numbers that don't fit in a standard integer type, which is exactly the scenario this problem is designed to represent.

Why We Optimize NEW

Since a carry can potentially ripple all the way from the last digit back to the first (like `999` becoming `1000`), the natural approach is to **reverse the list, add 1 with carry propagation from the new front (originally the last digit), then reverse back** — or equivalently, use recursion to process from the last node first (unwinding back to the front, similar to Recursive Insertion Sort's structure) so the carry naturally propagates backward through the return values. Either way, this handles arbitrarily large numbers correctly in O(n) time without ever converting to a fixed-size integer type that could overflow.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Add 1 to a number represented by LinkedList.

Visual Diagram

```
List: [1] → [2] → [3] (represents 123)
       Add 1: [1] → [2] → [4] (124)

       List: [9] → [9] → [9] (999)
       Add 1: [1] → [0] → [0] → [0] (1000)

       Method: Reverse and add
       Reverse: [3] → [2] → [1] → null
```

```
Add 1: carry=1
3+1=4, carry=0
2+0=2
1+0=1
Reverse back: [1] → [2] → [4]

Alternative: Recursive
AddOne(head):
  If head == null: Return 1
  carry = AddOne(head.next)
  If carry == 0: Return 0
  head.data += 1
  If head.data == 10:
    head.data = 0
    Return 1
  Return 0
```

Solution Approach

Concept: Use recursion or reverse and add.

Algorithm

```
Algorithm ADD_ONE(head):
           carry = ADD_CARRY(head)
           If carry == 1:
               newNode = Node(1)
               newNode.next = head
               Return newNode
           Return head

       Algorithm ADD_CARRY(node):
           If node == null:
               Return 1
           carry = ADD_CARRY(node.next)
           If carry == 0:
               Return 0
           node.data += 1
           If node.data == 10:
               node.data = 0
               Return 1
           Return 0
```

Java Implementation (Recursive)

class Solution {

public Node addOne(Node head) {

int carry = addCarry(head);

```
if (carry == 1) {
Node newNode = new Node(1);
newNode.next = head;
return newNode;
}
return head;
}
private int addCarry(Node node) {
if (node == null) {
return 1;
}
int carry = addCarry(node.next);
if (carry == 0) {
return 0;
}
node.data += 1;
if (node.data == 10) {
node.data = 0;
return 1;
}
return 0;
}
}
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n) recursion stack

118. Add 2 Numbers in LL

Understanding the Problem NEW

Two numbers are each represented as linked lists of digits (commonly stored in reverse order, so the head is the least significant digit), and you must add them together and return the sum as a new linked list, again handling carries correctly.

Brute Force Approach NEW

Converting both linked lists to actual integers, adding them normally, and converting the result back to a linked list works for small numbers but fails the same way as problem 117 for very large numbers that overflow standard integer types.

Why We Optimize NEW

Walk both lists simultaneously from their heads (least significant digits, if stored in reverse order), adding corresponding digits together along with any carry from the previous position, creating a new result node for each position's digit, and continuing past the end of the shorter list if one list is longer than the other (treating missing digits as 0). Any leftover carry after both lists are exhausted becomes one final extra digit. This is a direct digit-by-digit simulation of elementary-school long addition, run in O(max(n,m)) time — never needing to represent the whole number as a single integer at once, which is exactly why linked-list digit representation is used for arbitrarily large numbers in the first place.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Add two numbers represented by LinkedLists.

Visual Diagram

```
List 1: [2] → [4] → [3] (243)
       List 2: [5] → [6] → [4] (564)
       Result: [8] → [0] → [7] (807)

       Step-by-step:
       2 + 5 = 7 → carry 0
```

```
4 + 6 = 10 → digit 0, carry 1
3 + 4 + 1 = 8 → digit 8, carry 0

List 1: [9] → [9] (99)
List 2: [1] (1)
Result: [1] → [0] → [0] (100)

9 + 1 = 10 → digit 0, carry 1
9 + 0 + 1 = 10 → digit 0, carry 1
carry = 1 → new node [1]
```

Solution Approach

Concept: Traverse and add digit by digit with carry.

Algorithm

```
Algorithm ADD_TWO_NUMBERS(l1, l2):
           dummy = Node(0)
           current = dummy
           carry = 0

           While l1 != null or l2 != null or carry > 0:
               sum = carry
               If l1 != null:
                   sum += l1.data
                   l1 = l1.next
               If l2 != null:
                   sum += l2.data
                   l2 = l2.next

                 carry = sum / 10
                 current.next = Node(sum % 10)
                 current = current.next

           Return dummy.next
```

Java Implementation

```
class Solution {
           public Node addTwoNumbers(Node l1, Node l2) {
               Node dummy = new Node(0);
               Node current = dummy;
               int carry = 0;

                 while (l1 != null || l2 != null || carry > 0) {
                     int sum = carry;

                    if (l1 != null) {
```

```
                    sum += l1.data;
                    l1 = l1.next;
                }
                if (l2 != null) {
                    sum += l2.data;
                    l2 = l2.next;
                }

                carry = sum / 10;
                current.next = new Node(sum % 10);
                current = current.next;
            }

            return dummy.next;
        }
    }
```

Time & Space Complexity

Time Complexity: O(max(n,m)) | Space Complexity: O(max(n,m))

119. Delete All Occurrences of a Key in DLL

Understanding the Problem NEW

Delete every node containing a specific key value from a Doubly LinkedList (there may be multiple occurrences, unlike a simple single-delete).

Brute Force Approach NEW

Convert to an array, filter out matching values, and rebuild the list from scratch — O(n) time but O(n) extra space and discards all existing nodes unnecessarily.

Why We Optimize NEW

Because DLL nodes have direct access to both their previous and next neighbours, you can walk through the list once and, whenever you find a matching node, immediately unlink it in O(1) by connecting its neighbours directly to each other (using the same technique as problem 102) — no need to track a separate 'previous' pointer manually the way you would in a singly linked list, since each node already knows its own predecessor. This deletes all occurrences in a single O(n) pass with O(1) extra space, reusing DLL's key advantage directly.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Delete all nodes with a specific key in Doubly LinkedList.

Visual Diagram

```
Original: null ← [1] ↔ [2] ↔ [3] ↔ [2] ↔ [4] → null
       Delete key = 2

       Step 1: Check head
       head=1 → not delete
       Step 2: Check 2 → delete
       1.next = 3, 3.prev = 1
       Step 3: Check 3 → not delete
       Step 4: Check 2 → delete
       3.next = 4, 4.prev = 3
       Step 5: Check 4 → not delete
```

```
Result: null ← [1] ↔ [3] ↔ [4] → null
```

Solution Approach

Concept: Traverse and delete matching nodes.

Algorithm

```
Algorithm DELETE_KEY_DLL(head, key):
           current = head
           While current != null:
               If current.data == key:
                   If current == head:
                       head = head.next
                       If head != null: head.prev = null
                   Else:
                       current.prev.next = current.next
                       If current.next != null:
                           current.next.prev = current.prev
               current = current.next
           Return head
```

Java Implementation

```
class Solution {
           public DLLNode deleteAllOccurrences(DLLNode head, int key) {
               DLLNode current = head;

                 while (current != null) {
                     if (current.data == key) {
                         if (current == head) {
                             head = head.next;
                             if (head != null) {
                                  head.prev = null;
                             }
                         } else {
                             current.prev.next = current.next;
                             if (current.next != null) {
                                  current.next.prev = current.prev;
                             }

                        }
                    }
                    current = current.next;
               }
               return head;
           }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

120. Find Pairs with Given Sum in DLL

Understanding the Problem NEW

Given a **sorted** Doubly LinkedList, find all pairs of nodes whose values sum to a given target — the linked-list version of the Two Sum problem, but with a sorted, bidirectional structure available instead of a hashmap.

Brute Force Approach NEW

Just like Two Sum on an unsorted array, you could use a hashset of values seen so far as you traverse once — O(n) time, O(n) extra space — but this doesn't take advantage of the list being sorted and bidirectional.

Why We Optimize NEW

Because the DLL is sorted *and* supports backward traversal, you can apply the exact **two-pointer** technique used for sorted arrays in problems like 3-Sum: one pointer starts at the head, one at the tail, and you move them toward each other based on whether the current pair's sum is too small or too large. This achieves O(n) time with O(1) extra space (no hashset needed), directly exploiting the DLL's sortedness and backward pointers — something a singly linked list couldn't offer as cleanly.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find all pairs in sorted DLL that sum to target.

Visual Diagram

```
List: null ← [1] ↔ [2] ↔ [3] ↔ [4] ↔ [5] → null
       Target: 5

       Pairs:
       1 + 4 = 5 ✓
       2 + 3 = 5 ✓

       Step 1: left = head (1), right = tail (5)
       Sum = 6 > 5 → right--
       Sum = 1+4=5 → pair found, left++, right--
       Sum = 2+3=5 → pair found, left++, right--
```

```
       Result: (1,4), (2,3)
```

Solution Approach

Concept: Two pointers from both ends.

Algorithm

```
Algorithm PAIRS_IN_DLL(head, target):
           left = head
           right = head
           While right.next != null:
               right = right.next

            pairs = []
            While left != right and left.prev != right:
                sum = left.data + right.data
                If sum == target:
                    pairs.add([left.data, right.data])
                    left = left.next
                    right = right.prev
                Else If sum < target:
                    left = left.next
                Else:
                    right = right.prev
            Return pairs
```

Java Implementation

```
class Solution {
           public List<int[]> findPairs(DLLNode head, int target) {
               List<int[]> pairs = new ArrayList<>();
               DLLNode left = head;
               DLLNode right = head;

                 while (right.next != null) {
                     right = right.next;
                 }

                 while (left != null && right != null && left != right &&
left.prev != right) {
                     int sum = left.data + right.data;

                     if (sum == target) {

                      pairs.add(new int[]{left.data, right.data});
                      left = left.next;
                      right = right.prev;
                  } else if (sum < target) {
```

```
                    left = left.next;
                } else {
                    right = right.prev;
                }
            }
            return pairs;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

121. Remove Duplicates from Sorted DLL

Understanding the Problem NEW

Given a sorted Doubly LinkedList, remove any duplicate values, keeping only one copy of each distinct value.

Brute Force Approach NEW

Convert to an array, deduplicate (as in the Arrays-chapter Remove Duplicates problem), then rebuild the list — O(n) time but O(n) extra space and needless node reconstruction.

Why We Optimize NEW

Because the list is sorted, duplicates are always adjacent — so you can walk through the list once, and whenever the current node's value equals the next node's value, unlink that next node directly (an O(1) DLL deletion, same as problem 102) instead of advancing past it. This is a direct linked-list translation of the two-pointer 'remove duplicates from sorted array' technique (problem 11), achieving O(n) time and O(1) extra space by rewiring pointers instead of copying to an array.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Remove duplicate nodes from sorted Doubly LinkedList.

Visual Diagram

```
Original: null ← [1] ↔ [2] ↔ [2] ↔ [3] ↔ [3] ↔ [4] → null
       Result: null ← [1] ↔ [2] ↔ [3] ↔ [4] → null

       Step-by-step:
       current = 1
       next = 2 (1 != 2)
       current = 2
       next = 2 (duplicate)
       Skip duplicate: 2.next = 3, 3.prev = 2
       current = 2
       next = 3 (2 != 3)
       current = 3
```

```
        next = 3 (duplicate)
        Skip duplicate: 3.next = 4, 4.prev = 3
        current = 3
        next = 4 (3 != 4)
        current = 4
        next = null (done)
```

Solution Approach

Concept: Traverse and skip duplicates.

Algorithm

```
Algorithm REMOVE_DUPLICATES_DLL(head):
           current = head
           While current != null and current.next != null:
               If current.data == current.next.data:
                   duplicate = current.next
                   current.next = duplicate.next
                   If duplicate.next != null:
                       duplicate.next.prev = current
               Else:
                   current = current.next
           Return head
```

Java Implementation

```
class Solution {
           public DLLNode removeDuplicates(DLLNode head) {
               if (head == null) return null;

                 DLLNode current = head;

                 while (current != null && current.next != null) {
                     if (current.data == current.next.data) {
                         DLLNode duplicate = current.next;
                         current.next = duplicate.next;
                         if (duplicate.next != null) {
                              duplicate.next.prev = current;
                         }
                     } else {
                         current = current.next;
                     }

              }
              return head;
          }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

122. Reverse LL in Groups of Given Size K

Understanding the Problem NEW

Reverse a linked list not all at once, but in chunks of size K — e.g. reversing `[1,2,3,4,5]` in groups of 2 gives `[2,1,4,3,5]` (the last group, if shorter than K, may or may not be reversed depending on the exact problem variant).

Brute Force Approach NEW

A beginner might extract each group of K nodes' values into a temporary array, reverse the array, and write the values back into the same nodes — this works (O(n) time) but relies on overwriting values rather than truly rewiring the list's node structure, which misses the point of practicing pointer manipulation.

Why We Optimize NEW

This combines two ideas you already have: the **three-pointer reversal** technique from problem 105, applied to just the current group of K nodes at a time, and **recursion** (or an iterative loop) to move on to the next group once the current one is reversed, connecting the reversed group's new tail to the result of reversing the next group. This is a genuine pointer-rewiring solution, still O(n) time overall (each node is only touched a constant number of times), reusing the core reversal building block from earlier in the chapter rather than treating each group as a separate array-conversion problem.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Reverse LinkedList in groups of K nodes.

Visual Diagram

```
List: [1] → [2] → [3] → [4] → [5] → [6] → [7] → null, K=3

       Group 1: [1,2,3] → [3,2,1]
       Group 2: [4,5,6] → [6,5,4]
       Group 3: [7] → [7]

       Result: [3] → [2] → [1] → [6] → [5] → [4] → [7] → null
```

```
Step-by-step:
Step 1: Reverse first K nodes
[3] → [2] → [1] → [4] → [5] → [6] → [7] → null

Step 2: Reverse next K nodes
[3] → [2] → [1] → [6] → [5] → [4] → [7] → null

Step 3: Last group (K=1) remains same
[3] → [2] → [1] → [6] → [5] → [4] → [7] → null
```

Solution Approach

Concept: Recursively reverse groups.

Algorithm

```
Algorithm REVERSE_K_GROUPS(head, K):
           current = head
           count = 0

           // Check if K nodes exist
           While current != null and count < K:
               current = current.next
               count++

           If count < K:
               Return head

           // Reverse first K nodes
           prev = null
           current = head
           count = 0
           While current != null and count < K:
               next = current.next
               current.next = prev
               prev = current
               current = next
               count++

           // Recursively reverse rest
           head.next = REVERSE_K_GROUPS(current, K)
           Return prev
```

Java Implementation

```
class Solution {
           public Node reverseKGroup(Node head, int k) {
               Node current = head;
               int count = 0;
```

```
            // Check if enough nodes
            while (current != null && count < k) {
                current = current.next;
                count++;
            }
            if (count < k) {
                return head;
            }

            // Reverse first k nodes
            Node prev = null;
            current = head;
            count = 0;
            while (current != null && count < k) {
                Node next = current.next;
                current.next = prev;
                prev = current;
                current = next;
                count++;
            }

            // Recursively reverse rest
            head.next = reverseKGroup(current, k);
            return prev;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n/k) recursion stack

123. Rotate a LinkedList

Understanding the Problem NEW

Rotate a linked list to the right by K positions — e.g. rotating `[1,2,3,4,5]` right by 2 gives `[4,5,1,2,3]`. This is the linked-list equivalent of the array rotation problems (12-13) from Chapter 2.

Brute Force Approach NEW

A beginner might repeatedly move the last node to the front, one rotation at a time, K times — each single rotation requires walking to the end of the list (O(n)), so doing this K times costs O(n×K) total, mirroring the naive 'rotate by one, repeated' approach from the array version.

Why We Optimize NEW

You can do this in a single pass: first find the list's length and connect the tail back to the head temporarily (forming a circle), then walk forward to the position that will become the **new head** (which is `length - (K % length)` steps from the original head — using modulo in case K is larger than the list's length), and break the circle at exactly that point. This achieves the rotation in O(n) time regardless of how large K is, instead of O(n×K), by directly computing the new head position rather than simulating each individual rotation step.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Rotate LinkedList by K positions to the right.

Visual Diagram

```
List: [1] → [2] → [3] → [4] → [5] → null, K=2

       Step 1: Find length = 5
       Step 2: K = 2 % 5 = 2
       Step 3: Find new head position = 5 - 2 = 3
       Step 4: Traverse to node at position 3 (node 3)
       Step 5: Break at position 3, connect tail to head

       Original: [1] → [2] → [3] → [4] → [5] → null
                        ↓
```

```
       Break at 3: [1] → [2] → [3] → null
                                [4] → [5] → null
                        ↓
       Connect: [4] → [5] → [1] → [2] → [3] → null

       Result: [4] → [5] → [1] → [2] → [3] → null
```

Solution Approach

Concept: Find new head and reconnect.

Algorithm

```
Algorithm ROTATE_LL(head, k):
           If head == null: Return null

            // Find length
            tail = head
            len = 1
            While tail.next != null:
                tail = tail.next
                len++

            k = k % len
            If k == 0: Return head

            // Find new head
            steps = len - k
            current = head
            For i = 1 to steps-1:
                current = current.next

            // Rotate
            newHead = current.next
            current.next = null
            tail.next = head

            Return newHead
```

Java Implementation

```
class Solution {
           public Node rotateRight(Node head, int k) {
               if (head == null) return null;
               // Find length and tail

              Node tail = head;
              int len = 1;
              while (tail.next != null) {
                  tail = tail.next;
```

```
                len++;
            }

            k = k % len;
            if (k == 0) return head;

            // Find new head
            int steps = len - k;
            Node current = head;
            for (int i = 1; i < steps; i++) {
                current = current.next;
            }

            // Rotate
            Node newHead = current.next;
            current.next = null;
            tail.next = head;

            return newHead;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

124. Flattening of LinkedList

Understanding the Problem NEW

Each node in this linked list can have not just a 'next' pointer, but also a 'child' pointer to a separate sub-list, which can itself have further nested sub-lists (multiple levels deep, like a tree of linked lists). Flatten this whole structure into one single-level linked list, in the correct visiting order.

Brute Force Approach NEW

A beginner might try to handle this with ad-hoc, deeply nested loops trying to track 'depth' manually — this quickly becomes unmanageable and error-prone as the nesting depth grows unpredictably.

Why We Optimize NEW

This structure is naturally suited to a **depth-first, recursive** approach: whenever you encounter a node with a child pointer, recursively flatten that child sub-list *first*, then splice the fully-flattened child list in between the current node and whatever it originally pointed to next, before continuing the traversal. Recursion elegantly handles the arbitrary, unpredictable nesting depth without you needing to manually track how deep you are — each level 'trusts' the recursive call to fully flatten what's below it, similar in spirit to how divide-and-conquer trusts smaller subproblems to already be solved.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Flatten a multilevel LinkedList into a single level.

Visual Diagram

```
Original:
       1 → 2 → 3 → 4 → 5 → 6 → null
           ↓
           7 → 8 → 9 → null
               ↓
               10 → 11 → null

       Flattened:
```

```
1 → 2 → 7 → 8 → 10 → 11 → 9 → 3 → 4 → 5 → 6 → null

Approach:
1. Start from head (1)
2. Traverse to tail (6)
3. Connect tail to child head (7)
4. Continue until all flattened
```

Solution Approach

Concept: Use DFS to flatten recursively.

Algorithm

```
Algorithm FLATTEN_LL(head):
           current = head

            While current != null:
                If current.child != null:
                    // Find tail of child list
                    tail = current.child
                    While tail.next != null:
                        tail = tail.next

                     // Connect tail to current.next
                     tail.next = current.next
                     if current.next != null:
                         current.next.prev = tail

                     // Connect current to child
                     current.next = current.child
                     current.child.prev = current
                     current.child = null

                 current = current.next

            Return head
```

Java Implementation

```
class Node {
           int data;
           Node next;
           Node prev;
           Node child;

            Node(int data) {

              this.data = data;
         }
```

```
}

class Solution {
    public Node flatten(Node head) {
        Node current = head;

        while (current != null) {
            if (current.child != null) {
                // Find tail of child
                Node tail = current.child;
                while (tail.next != null) {
                    tail = tail.next;
                }

                // Connect tail to current.next
                tail.next = current.next;
                if (current.next != null) {
                    current.next.prev = tail;
                }

                // Connect current to child
                current.next = current.child;
                current.child.prev = current;
                current.child = null;
            }
            current = current.next;
        }
        return head;
    }
}
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

125. Clone a Linked List with Random and Next Pointers

Understanding the Problem NEW

Each node has a normal 'next' pointer, but also a 'random' pointer that can point to *any* other node in the list (or null) — arbitrarily, not necessarily in sequence. Create a complete, independent deep copy of this list, correctly replicating both pointer types.

Brute Force Approach NEW

A beginner's first idea might use a hashmap from original nodes to their newly created clone nodes, built in one pass, then a second pass to wire up both the next and random pointers on the clones by looking up the corresponding clone in the hashmap — this works correctly in O(n) time, but costs O(n) extra space for the hashmap.

Why We Optimize NEW

There's a clever O(1)-extra-space trick: first, interleave a clone of each node directly *after* its original (so the list becomes original→clone→original→clone...); this lets you set each clone's 'random' pointer easily, since `original.random.next` is exactly that random target's clone (because every clone was placed right after its original); finally, in a third pass, carefully unweave the interleaved list back into two separate lists — the original (restored) and the fully-wired clone. This achieves the same O(n) time as the hashmap approach, but O(1) extra space, by using the list's own structure as temporary storage instead of an external map.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Clone LinkedList with next and random pointers.

Visual Diagram



```
        ■■■random■■→    ■■■random■■→   ■■■random■■→       ■■■random■■→

       Approach 1: HashMap
       Step 1: Create mapping old → new
       [1] → [1'], [2] → [2'], [3] → [3'], [4] → [4']

       Step 2: Copy random pointers
       [1'].random = map.get([1].random)

       Approach 2: Interleaving
       Step 1: Insert clone nodes
       [1] → [1'] → [2] → [2'] → [3] → [3'] → [4] → [4']

       Step 2: Set random pointers
       [1'].random = [1].random.next

       Step 3: Extract cloned list
       [1'] → [2'] → [3'] → [4']
```

Solution Approach

Concept: Three-step interleaving approach.

Algorithm

```
Algorithm CLONE_RANDOM_LL(head):
           If head == null: Return null

           // Step 1: Insert clone nodes
           current = head
           While current != null:
               clone = Node(current.data)
               clone.next = current.next
               current.next = clone
               current = clone.next

           // Step 2: Set random pointers
           current = head
           While current != null:
               If current.random != null:
                   current.next.random = current.random.next
               current = current.next.next

           // Step 3: Extract cloned list
           dummy = Node(0)
           clone = dummy
           current = head
           While current != null:
               clone.next = current.next
               clone = clone.next
               current.next = clone.next
```

```
                current = current.next

            Return dummy.next
```

Java Implementation

```
class Node {
            int data;
            Node next;
            Node random;

            Node(int data) {
                this.data = data;
            }
        }

        class Solution {
            public Node copyRandomList(Node head) {
                if (head == null) return null;

                // Step 1: Insert clone nodes
                Node current = head;
                while (current != null) {
                    Node clone = new Node(current.data);
                    clone.next = current.next;
                    current.next = clone;
                    current = clone.next;
                }

                // Step 2: Set random pointers
                current = head;
                while (current != null) {
                    if (current.random != null) {
                        current.next.random = current.random.next;
                    }
                    current = current.next.next;
                }

                // Step 3: Extract cloned list
                Node dummy = new Node(0);
                Node clone = dummy;
                current = head;
                while (current != null) {
                    clone.next = current.next;
                    clone = clone.next;
                    current.next = clone.next;
                    current = current.next;
                }

                return dummy.next;
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1) (excluding cloned list)

LinkedList Summary — Complexity Comparison

Problem	Type	Time	Space	Key Technique
Basic Operations	Easy	O(1)-O(n)	O(1)	Pointer manipulation
Middle Node	Medium	O(n)	O(1)	Tortoise-Hare
Reverse (Iterative)	Medium	O(n)	O(1)	Three pointers
Reverse (Recursive)	Medium	O(n)	O(n)	Recursion
Detect Cycle	Medium	O(n)	O(1)	Floyd's Algorithm
Cycle Start	Medium	O(n)	O(1)	Floyd's + reset
Palindrome	Medium	O(n)	O(1)	Reverse half
Odd-Even	Medium	O(n)	O(1)	Two lists
Remove Nth	Medium	O(n)	O(1)	Two pointers
Sort LL	Medium	O(n log n)	O(log n)	Merge sort
Sort 0,1,2	Medium	O(n)	O(1)	Dutch flag
Intersection	Medium	O(n+m)	O(1)	Length difference
Add Two Numbers	Medium	O(n)	O(n)	Digit addition
DLL Operations	Medium	O(n)	O(1)	Prev/Next
Reverse K Groups	Hard	O(n)	O(n/k)	Recursive groups
Rotate	Hard	O(n)	O(1)	Find new head
Flattening	Hard	O(n)	O(1)	DFS insertion
Clone with Random	Hard	O(n)	O(1)	Interleaving